

HealthPro — Фаза 2 + Етап 5

Завдання для Replit Agent: 3 фінальних баги + Capacitor APK + SQLite

Версія: 1.0 | Дата: 26 квітня 2026 р. | Фаза 1: 100% завершена, 41 тест зелений, CI OK

ЗАГАЛЬНИЙ ПЛАН ВИКОНАННЯ

Частина	Завдання	Час	Пріоритет
Частина 1	3 фінальних баги (sticky, PDF модалка, CSV)	1-2 год	СПОЧАТКУ
Частина 2	Фаза 2: Capacitor Android APK	1-2 дні	ПОТІМ
Частина 3	Етап 5: SQLite міграція	2-3 дні	ОДРАЗУ ПІСЛЯ
Правило	npm test після кожного кроку — 41/41 зелених	авто	ЗАВЖДИ

ЧАСТИНА 1 — 3 ФІНАЛЬНИХ БАГИ

Три залишкові баги після двох раундів BugFix. Виправити до початку Фази 2. Після кожного виправлення — npm test.

Бег 1 — Sticky навігація стрибає при скролі

Навігаційна панель вкладок повинна залишатись зафіксованою під хедером. Поточний фікс top:62px — статичне значення, не відповідає реальній висоті хедера. Результат: навігація стрибає при скролі на обох пристроях.

А

Виміряти реальну висоту хедера

- Відкрити DevTools → Inspector
- Клікнути на .header елемент
- Подивитись Computed → height (реальна висота в пікселях)
- Записати значення — воно може відрізнятись від 62px

Б	Виправити через CSS змінну • У styles/layout.css знайти <code>.nav-tabs { position: sticky; top: ... }</code> • Замінити статичне значення на: <code>top: var(--header-height, 62px)</code> • У styles/base.css додати: <code>:root { --header-height: 70px }</code> • Або виміряти через JS: <code>document.querySelector('.header').offsetHeight</code> • І встановити: <code>document.documentElement.style.setProperty('--header-height', height + 'px')</code>
В	JS підхід (надійніший для динамічного хедера) • У src/ui/ або src/app.js після рендеру хедера: • <code>const h = document.querySelector('.header')?.offsetHeight 70</code> • <code>document.documentElement.style.setProperty('--header-height', h + 'px')</code> • Викликати при: <code>DOMContentLoaded</code> + зміни мови + зміни теми
Г	Критерій завершення • Навігація не стрибає при скролі на мобільному і ноутбучі • Навігація щільно прилягає до хедера без зазору • <code>npm test</code> → 41/41 зелений

Баг 2 — Модалка не закривається після PDF друку

Після натискання 'Друк/PDF' у модальному вікні звіту: з'являється toast, генерується PDF, але модалка залишається відкритою. Користувач змушений закривати її вручну хрестиком.

А	Знайти функцію <code>printReportPeriod</code> • Відкрити <code>src/features/export/print.js</code> • Знайти функцію <code>printReportPeriod()</code> або <code>exportPDF()</code> • Перевірити чи є там виклик <code>closeExportModal()</code> після генерації
Б	Додати закриття модалки після завершення • У функції <code>printReportPeriod()</code> після успішного друку/збереження: • Додати: <code>closeExportModal()</code> або <code>emit('modal:close')</code> або • <code>document.getElementById('exportModal').classList.remove('show')</code> • Додати також у <code>exportPDF()</code> після збереження файлу • На мобільному: після <code>platform.download()</code> → закрити модалку
В	Критерій завершення • Після друку/збереження PDF модалка закривається автоматично • Toast 'Звіт збережено' відображається • <code>npm test</code> → 41/41 зелений

Баг 3 — CSV тиша при натисканні 'Зберегти CSV'

При натисканні 'Зберегти CSV' у модальному вікні звіту — нічого не відбувається. Файл не завантажується, toast не з'являється, краш відсутній. Попередній фікс (try/catch) прибрав краш але не виправив логіку.

А

Діагностика — відкрити DevTools Console

- Відкрити DevTools → Console
- Натиснути 'Зберегти CSV' у модальці
- Записати точне повідомлення з консолі якщо є
- Також перевірити вкладку Network — чи є якийсь запит

Б

Перевірити функцію exportReportCSV

- Відкрити src/features/export/csv.js
- Знайти функцію exportReportCSV()
- Перевірити: чи коректно визначається period (week/2weeks/month/custom)
- Перевірити: чи є дані у state.measurements для вибраного періоду
- Додати console.log на початку функції для діагностики

В

Типові причини тиші

- Blob URL не створюється через відсутність даних → додати перевірку
- measurements.filter() повертає порожній масив → показати toast 'Немає даних'
- Елемент expDateFrom/expDateTo не знайдено → optional chaining
- Функція не зареєстрована як обробник події → перевірити app.js

Г

Критерій завершення

- CSV файл завантажується при наявності даних
- При відсутності даних показується toast з поясненням
- Модалка закривається після успішного збереження
- npm test → 41/41 зелений

ЧАСТИНА 2 — ФАЗА 2: CAPACITOR ANDROID APK

Перша нативна збірка Android. Застосунок використовує `localStorage` — це нормально для першого APK. `SQLite` додамо одразу після (Частина 3). Мета: тримати в руках реальний APK і знайти нативні баги.

2.1 Підготовка та ініціалізація Capacitor

1. Перевірити `capacitor.config.json`
 - Відкрити `capacitor.config.json` у корені репо
 - Переконались: `{ "appId": "ua.healthpro.app", "appName": "HealthPro", "webDir": "dist" }`
 - `webDir` має бути `'dist'` — не абсолютний шлях!
2. Встановити Capacitor плагіни
 - `cd HealthPro-Moie-Zdorovia`
 - `npm install @capacitor/local-notifications`
 - `npm install @capacitor/share`
 - `npm install @capacitor/preferences`
 - `npm install @capacitor/filesystem`
 - `npm install @capacitor/haptics`
 - `npm install @capacitor/status-bar`
 - `npm install @capacitor/splash-screen`
 - `npm install @capacitor/app`
3. Зробити білд та синхронізувати
 - `npm run build`
 - `npx cap sync android`
 - Якщо `android/` папка відсутня: `npx cap add android`
 - Перевірити що `dist/` папка з'явилась після `build`

2.2 Оновити `platform.js` для нативних плагінів

`core/platform.js` вже існує з `web-fallback`. Тепер підключити реальні Capacitor плагіни замість заглушок.

4. Оновити нотифікації
 - Замінити `web Notification API` на `@capacitor/local-notifications`
 - `import { LocalNotifications } from '@capacitor/local-notifications'`
 - `requestNotificationPermission(): await LocalNotifications.requestPermissions()`
 - `notify(title, body): await`
`LocalNotifications.schedule({notifications:[{title,body,id:Date.now()}]})`

5

Оновити Share та Download

- `import { Share } from '@capacitor/share'`
- `share(data)`: якщо `isNative()` → `await Share.share(data)`
- `import { Filesystem, Directory } from '@capacitor/filesystem'`
- `download(filename, blob)`: якщо `isNative()` → `Filesystem.writeFile + Share.share`

6

Оновити Preferences (замість `localStorage`)

- `import { Preferences } from '@capacitor/preferences'`
- `prefs.get(key)`: якщо `isNative()` → `(await Preferences.get({key})).value`
- `prefs.set(key, val)`: якщо `isNative()` → `Preferences.set({key, value: val})`
- Це підготовка до SQLite — `localStorage` залишається як `web-fallback`

2.3 Збірка APK та тестування

7

Відкрити Android Studio

- `nx cap open android`
- Android Studio відкриє проект автоматично
- Зачекати поки Gradle синхронізується (може зайняти 5-10 хв)

8

Зібрати debug APK

- Build → Build Bundle(s) / APK(s) → Build APK(s)
- APK буде у: `android/app/build/outputs/apk/debug/app-debug.apk`
- Перенести на телефон та встановити

9

Тестувати на реальному пристрої

- Перевірити: всі вкладки відкриваються
- Перевірити: збереження вимірювань тиску
- Перевірити: нотифікації (дозвіл + тест)
- Перевірити: PDF/CSV експорт через Share
- Перевірити: перемикання теми
- Зафіксувати всі знайдені баги у GitHub Issues

ЧАСТИНА 3 — ЕТАП 5: SQLite МІГРАЦІЯ

Одразу після першого робочого APK. SQLite замінює localStorage для надійного зберігання медичних даних. Підхід: прямі запити БЕЗ in-memory кешу на початку — кеш додамо тільки якщо виникне реальне гальмування.

3.1 Встановлення та схема

- 1 Встановити SQLite плагін
 - `npm install @capacitor-community/sqlite`
 - `npx cap sync android`
 - Створити файл `src/core/db/sqlite.js`
- 2 Схема бази даних — створити в `sqlite.js`
 - `CREATE TABLE IF NOT EXISTS measurements (id INTEGER PRIMARY KEY AUTOINCREMENT, ts INTEGER NOT NULL, sys INTEGER NOT NULL, dia INTEGER NOT NULL, pulse INTEGER, note TEXT)`
 - `CREATE TABLE IF NOT EXISTS pills (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, dose TEXT, time TEXT, days TEXT, date INTEGER)`
 - `CREATE TABLE IF NOT EXISTS pills_taken (date TEXT NOT NULL, pill_id INTEGER NOT NULL, taken INTEGER NOT NULL DEFAULT 0, PRIMARY KEY (date, pill_id))`
 - `CREATE TABLE IF NOT EXISTS settings (key TEXT PRIMARY KEY, value TEXT)`
 - `CREATE INDEX IF NOT EXISTS idx_measurements_ts ON measurements(ts DESC)`

3.2 API `sqlite.js`

- 3 Створити `core/db/sqlite.js` з методами
 - `open()`: відкрити/створити БД, виконати `CREATE TABLE IF NOT EXISTS`
 - `run(sql, params)`: виконати `INSERT/UPDATE/DELETE`
 - `get(sql, params)`: отримати один рядок
 - `all(sql, params)`: отримати масив рядків
 - Web-fallback: якщо `!isNative()` → використати `sql.js` через `IndexedDB`
 - Або простіший web-fallback: зберігати у `localStorage` як JSON (тимчасово)

3.3 Міграція `localStorage` → `SQLite`

- 4 **ОБОВ'ЯЗКОВО:** резервна копія перед міграцією
 - Перед будь-якими змінами: `exportData()` → зберегти JSON файл
 - Показати користувачу прогрес-бар під час міграції
 - Після міграції: порівняти кількість записів у `localStorage` та `SQLite`

5

Порядок міграції

- 1. Прочитати всі measurements з localStorage
- 2. Вставити у SQLite через INSERT INTO measurements
- 3. Прочитати всі pills та pills_taken
- 4. Вставити у відповідні таблиці
- 5. Перенести settings (мова, тема, профіль)
- 6. Позначити міграцію як виконану: settings.set('migrated_v1', 'true')

3.4 Оновлення модулів

6

Оновити по черзі — після кожного npm test

- pressure/index.js: saveMeasurement → db.run INSERT INTO measurements
- history/index.js: renderHistory → db.all SELECT FROM measurements
- meds/index.js: addPill → db.run INSERT INTO pills
- analytics/index.js: читати з db замість state.measurements
- Після кожного модуля: npm test → 41/41 зелений

7

Критерій завершення Етапу 5

- Всі дані зберігаються у SQLite на пристрої
- localStorage використовується тільки як web-fallback у браузері
- Експорт JSON/CSV читає дані з SQLite
- npm test → 41/41 зелений
- APK тест: дані зберігаються після перезапуску застосунку

ЗВЕДЕНА ТАБЛИЦЯ КРОКІВ

№	Дія	Час	Частина
1	Sticky навігація: JS вимір висоти хедера	30 хв	1 — Баги
2	Модалка: закрити після PDF/CSV	30 хв	1 — Баги
3	CSV: діагностика + виправити логіку	1 год	1 — Баги
4	npm test → 41/41, git push, CI зелений	10 хв	1 — Баги
5	capacitor.config.json перевірити webDir: dist	5 хв	2 — APK
6	npm install Capacitor плагіни	15 хв	2 — APK
7	npm run build + npx cap sync android	10 хв	2 — APK
8	Оновити platform.js: notifications, share, filesystem	2-3 год	2 — APK
9	Android Studio: зібрати debug APK	30 хв	2 — APK

10	Тест APK на реальному телефоні	1 год	2 — APK
11	npm install @capacitor-community/sqlite	15 хв	3 — SQLite
12	Створити core/db/sqlite.js: схема + API	3-4 год	3 — SQLite
13	Міграція localStorage → SQLite з резервною копією	2-3 год	3 — SQLite
14	Оновити pressure, history, meds, analytics	3-4 год	3 — SQLite
15	Фінальний тест APK з SQLite	1 год	3 — SQLite

ВАЖЛИВІ ЗАСТЕРЕЖЕННЯ

npm test після КОЖНОГО кроку
41/41 зелених — це єдиний критерій що код не зламано. Якщо тест впав — виправити одразу, не рухатись далі.

SQLite БЕЗ in-memory кешу на початку
Починати з прямих запитів до БД. Кеш додавати тільки якщо виникне реальне гальмування на пристрої. Передчасна оптимізація = зайва складність.

Тестувати APK на реальному пристрої
Емулятор не замінює реальний телефон для медичного застосунку. Нотифікації, PDF, Share — все тестувати на реальному Android.

Резервна копія ПЕРЕД міграцією SQLite
Обов'язково зберегти JSON-бекап всіх даних перед міграцією. Якщо щось піде не так — дані не загубляться.

git push після кожної частини
Частина 1 → commit + push. Частина 2 → commit + push. Частина 3 → commit + push. CI має залишатись зеленим після кожного пушу.